

Challenge @polito
By Storm AI Reply

Refhunters

SIMPLIFY HIDDEN SCIENTIFIC REFERENCES



**Politecnico
di Torino**



Our team

ZOHREH AKBARI

computer engineering

ANDREA OLIVERO

management engineering

NEGAR HOSSEINALI POUR

computer engineering

AMIRHESAM TORKHASVAND

computer engineering

SATHGURUNATHAN JAYACHANDRAN

Mechatronics Engineering



**Politecnico
di Torino**



CONNECTION LAB AND
INNOVATION KITCHEN

Overview

The goal is to develop an agentic system capable of generating reliable, in-depth answers from any scientific paper. Each response is transparently linked to verified text snippets and enriched through cited sources and context. In this WP₃ phase, the focus shifts to working prototype development, demonstrating key features, user interaction flow, and feedback gathering mechanisms.



**Politecnico
di Torino**



User Interface Design

1. Upload Page

allows users to upload a PDF or enter a DOI through a clean, minimal interface with a central search bar and quick action buttons.

2. Answer Page

offers a dual-pane workspace with the PDF on the left and the chat on the right. When users ask a question, the system returns an answer supported by precise excerpts from the paper—including page and paragraph numbers—with direct citation links. Each answer can be rated for reliability, enabling continuous improvement of the system's reasoning and agent behaviour.



Politecnico
di Torino



User Interaction Flow

Demo

<https://www.figma.com/make/9Wko9t4leKSWd2aQyuP0nU/Research-Assistant-App?node-id=0-1&p=f&t=YuWEOt6QwTjcphuu-0>

Key Features

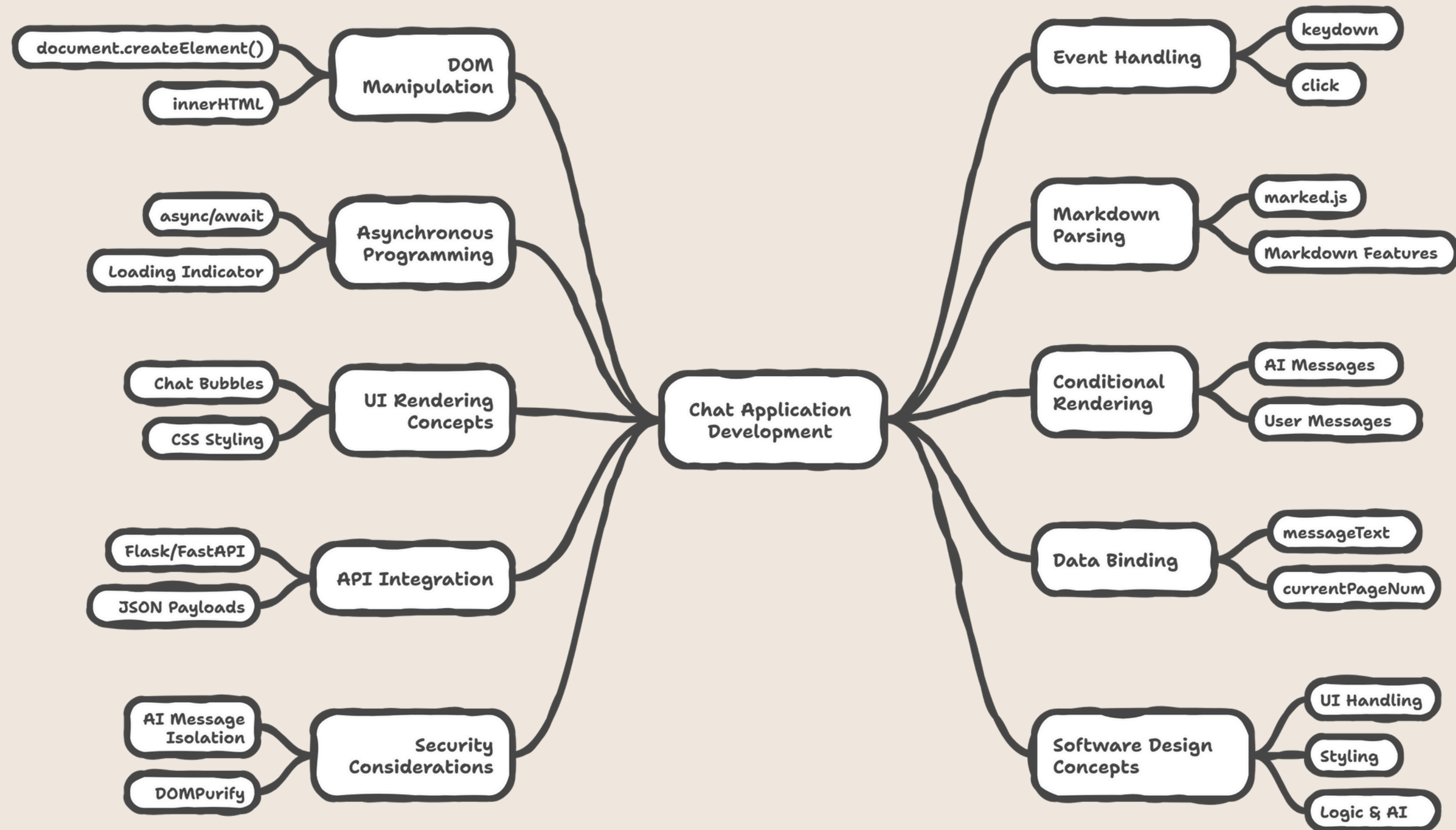
- Answer Complex Questions from Scientific Papers
- Seamless Multi-Agent Integration
- Citation Snippets
- Integrated PDF Viewer
- **Extra Feature:**
- Reliability Rating System



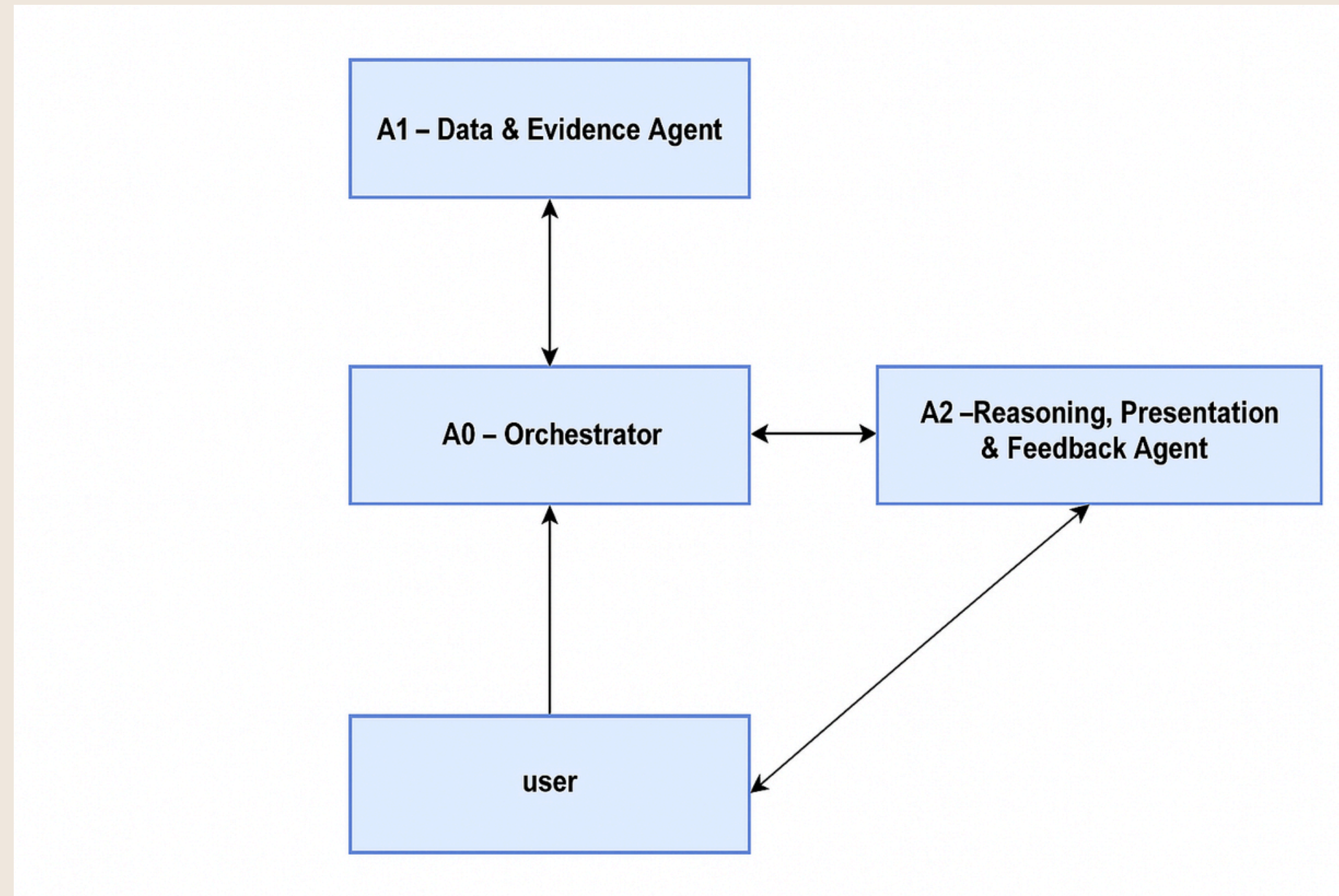
Politecnico
di Torino



Client Side: Key Concepts Used...



System Architecture Flow



System Goal

- Giving answers based on verified scientific sources.
- Ensure traceable snippets from papers & citations.
- Use a Knowledge Base to explore topics and relationships.

Architecture of A0 – Controller

A0 owns memory policy and lifecycle, while all agents can access memory through controlled interfaces.

Architecture of A0 – Controller

Entrance + Exit

Entrance:

Purpose: normalize input and hydrate global memory.

Exit:

Purpose: persist memory + produce final response payload.

Architecture of A0 – Controller

Small LLM Brain

Input:

user_message, memory summary

Output:

strict JSON (validated)

Architecture of A0 – Controller

Query Decomposer

enables parallel evidence retrieval and better coverage.

Architecture of A0 – Controller

Planning → TODO List

build a TODO list from

brain JSON + decomposed query + memory

Architecture of A0 – Controller

Dispatch

execute tasks with deps, parallelism, retries, timeouts.

A0 is the only component allowed to schedule/loop (agents don't self-loop).

Architecture of A0 – Controller

Evidence Quality Gate + Retry Loop

Compute risk_score from:

missing citations

contradictions across evidence

claims not mapped to evidence

Critical improvement: “Retry must change something”

Architecture of A0 – Controller

Get results from agents
& Human Feedback Loop

When returns feedback:

update Redis

apply immediate effects next run

agent 1 (*A1*)

core document-processing agent

agent 1 (*A1*)

$A0 \rightarrow A1$

Agent 1 only executes structured tasks sent from Agent 0.

e.g:

```
[“actions”: xxxx,  
“pdf_path”: xxxx,  
“keyword”: xxxx ]
```

agent 1 (A1)- Tools & Technologies

GROBID → Extracts citation

LangChain + ChatOpenAI → Converts raw XML into structured JSON

arXiv API → Searches papers, retrieves IDs, downloads PDFs

requests → HTTP API calls

re, json, os → Parsing & file management

dotenv → Loads API keys

agent 1 (A1)

Step 1 — PDF Processing with GROBID

- Input: Main scientific PDF
- GROBID extracts structured metadata
- Output: <biblStruct> citation blocks in TEI XML format.

JSON containing fields like title, authors, year, journal, and DOI.

agent 1 (A1)

Step 2 — XML → JSON Conversion (LLM via LangChain)

- Citation blocks sent to an LLM through LangChain
- LLM normalizes and formats each citation
- Output fields:
 - title
 - authors
 - year
 - journal
 - doi

agent 1 (A1)

- Step 3 — Keyword-Based Filtering
 - Agent 0 provides a keyword (e.g., “transformer”, “attention”)
 - Agent 1 filters citations whose titles match the keyword
 - Result: Only topic-related citations move to the next step
-
- Step 4 — arXiv Search
 - For each filtered citation:
 - Search arXiv using the citation title
 - Check if a corresponding paper exists
 - Output: arXiv ID (if found)

agent 1 (A1)

Step 5 — PDF Download or Abstract Fallback

If an arXiv ID exists:

Attempt to download the PDF

If download fails → retrieve the abstract from arXiv

Store results

Step 6 — Build Final Knowledge Base

Output: A structured, ready-to-use Knowledge Base

Q&A

We're happy to discuss your thoughts and feedback.



**Politecnico
di Torino**

